

Capire la comprehensions in Python

Introduzione

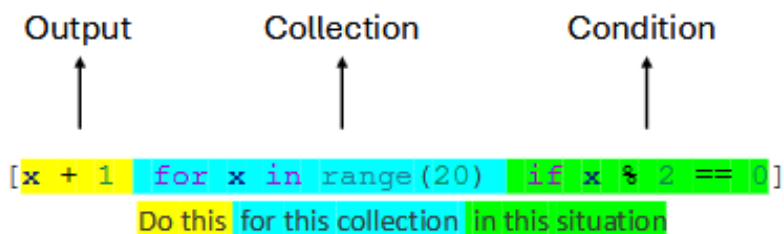
La **comprehension** in Python è una sintassi compatta per creare sequenze (liste, set, dizionari).

Anche se viene usata principalmente con le **liste**, può essere applicata a tutte le **collections**.

La comprehension ci permette di **popolare liste, set e dizionari** con una versione compatta di un for loop, scritta su una sola riga.

Sintassi

Le comprehension si scrivono così



sintax list comprehension.png

Guardando questa immagine:

- Le parentesi quadre indicano che il risultato sarà una lista.
- La parte in **giallo** rappresenta l'istruzione da eseguire: in questo caso `x + 1`.
- La parte in **azzurro** è il ciclo: per ogni `x` nel range da `0` a `19` (`range(20)`).
- La parte in **verde** è la condizione: esegui solo se `x` è **pari** (`x % 2 == 0`).

 **la condizione nelle list comprehension è opzionale**

Quindi questa comprehension ritornerà una lista di numeri **pari** da `0` a `19`, ognuno **aumentato di 1**.

Quindi per riassumere la sintassi di una comprehension è la seguente:

```
1 [expression for item in iterable if condition]
```

All'inizio (sinistra) l'espressione da eseguire
al centro si trova il for loop per iterare l'espressione
a destra vi è la condizione per cui se la condizione è vera la list comprehension verrà eseguita

Esempio 1: Aggiungere numeri pari a una lista



Python

```
1 squares = []
2 for x in range(10):
3     if x % 2 == 0:
4         squares.append(x**2)
5 #Con la comprehension
6 squares = [x**2 for x in range(10) if x % 2 == 0]
```

In questo esempio possiamo notare la differenza tra l'usare un `for` loop tradizionale e la list comprehension:

Funzionamento con il loop tradizionale:

1. Si crea una lista vuota: `squares = []`
2. Itera su `x` da 0 a 9 (`range(10)`)
3. Se `x` è pari (`x % 2 == 0`), aggiungi il quadrato di `x` alla lista (`squares.append(x**2)`).
4. Output: `[0, 14, 16, 36, 64]` (quadrati dei numeri pari 0,2,4,6,8).

Funzionamento con la list comprehension:

1. `x**2` : Output expression, cioè cosa inserire nella lista
2. `for x in range` : iterazione di `x` nel range numerico da 0 a 9.
3. `if x % 2 == 0` : Condizione, questa list comprehension verrà eseguita solo per i numeri pari.

Come possiamo notare nel funzionamento della list comprehension non si usa esplicitamente `.append()` poiché in questo caso la sintassi delle parentesi quadre `[]` si occupa implicitamente di restituire una lista e di aggiungere automaticamente gli elementi senza il bisogno di doverli "appendere".

Quindi perché non serve `.append()` nella list comprehension?

La list comprehension viene definita zucchero sintattico (syntactic sugar): la si usa per un pattern comune:

- Crea una lista -> esegue un loop-> verifica una condizione -> infine aggiunge uno o più elementi.
- Le parentesi quadre quindi in questo caso dicono a Python:
"Costruisci una lista con tutti gli `x**2` dove `x` è pari, senza che io debba scrivere manualmente l' `.append()` "

☰ Analogia Pratica

Se può essere d'aiuto, pensa a quando si ordina un caffè:

- **Senza comprehension:**
sostanzialmente dici a Python: "Prendi una tazza vuota, versa il caffè, aggiungi zucchero, mescola".
Quindi devi specificare i passaggi espliciti con il `for` + `.append()` .
- **Con comprehension:**
"Un caffè zuccherato, grazie".
In questo caso Python capisce al volo i passaggi impliciti, come le `[]`

❓ Si può usare la list comprehension per aggiungere elementi a una lista esistente?

Sì, ma ci sono alcune differenze concettuali importanti rispetto all'uso di `.append()` o `.extend()` :

☰ Caso 1: Creazione/Copia di liste ▾

Metodo tradizionale (con `append` o `extend`)



Python

```
1 lista_vuota = []
2 lista_piena = [1, 2, 3, 4]
3
4 # Opzione 1: Aggiunta elemento per elemento (con append)
5 for x in lista_piena:
6     lista_vuota.append(x) # Risultato: [1, 2, 3, 4]
7
8 # Opzione 2: Aggiunta diretta (con extend)
9 lista_vuota.extend(lista_piena) # Risultato: [1, 2, 3, 4]
```

Metodo con la list comprehension



Python

```
1 lista_vuota = []
2 lista_piena = [1, 2, 3, 4]
3
4 # List comprehension che "copia" gli elementi
5 lista_vuota = [x for x in lista_piena] # Risultato: [1,
    2, 3, 4]
```

🔔 **Attenzione:** Nell'esempio con la comprehension non si sta modificando la `lista_vuota`, come nell'esempio con l' `append` o l' `extend`, ma la si sta sovrascrivendo con una nuova lista.

Difatti se `lista_vuota` aveva altri elementi prima, questi vengono persi (a differenza di `append` / `extend`).

Quindi se si vuole preservare il contenuto originale di una lista vuota o di una lista con elementi già esistenti, o se si vuole aggiungere elementi alla lista, non si può usare direttamente la **list comprehension**, in questo caso è meglio usare l' `append` o l' `extend`. Quindi la list comprehension si usa quando si vuole creare una nuova lista o sovrascrivere il contenuto di una già esistente

Lettura ottimale delle List Comprehension

In genere le list comprehension sono comode da leggere **partendo dal centro** (cioè dal ciclo `for`), poiché questo riflette l'ordine logico di esecuzione.

Le list comprehension hanno una struttura a "cipolla" (dall'interno verso l'esterno), come dimostra l'[esempio di prima](#):



Python

```
1 [x**2 for x in range(10) if x % 2 == 0]
```

Flusso di lettura consigliato

La lettura ottimale segue questo flusso:

1. **Centro(nucleo):** `for x in range(10)` , cioè "per ogni iterazione di `x` da 0 a 9..."
 2. **Condizione (se presente):** `if x % 2 == 0` , cioè "...solo se `x` è pari..."
 3. **Output(risultato):** `x**2` , cioè "...calcola il quadrato di `x` "
- Quindi la **traduzione completa** è: "Per ogni iterazione di `x` da 0 a 9, se `x` è pari, calcola il quadrato di `x` ."

Questo flusso di lettura è molto più comodo per vari motivi:

1. Ordine logico:

- Prima devo chiedermi "Dove prendo i dati"? Dal ciclo `for`
- Poi "**Quali filtri/criteri applico?**" La condizione `if` , in questo caso: `if x % 2 == 0`
- Infine "**Cosa voglio ottenere?**" L'espressione `x**2`

2. Evita ambiguità:

intuitivamente viene da leggere partendo dall'output (`x**2`) prima del ciclo, tuttavia questo è un errore molto comune anche perché viene poi da chiedersi "*Ma da dove viene questa espressione (o output)?*". Quindi partire dal centro risolve l'ambiguità e chiarisce subito il contesto.

Esempio 2 : Convertire una stringa in uppercase

Ipotizziamo un caso in cui si abbia una lista di stringhe e si vuole visualizzare in output le stringhe in maiuscolo:



Python

```
1 names:list[str] = ["Alice", "bob", "carol"]
2 uppercase_names = [name.upper() for name in names]
3 print(uppercase_names)
```

Abbiamo la lista `names` che contiene 3 nomi con varie capitalizzazioni:

- "Alice" (maiuscolo iniziale)
- "bob" (tutto minuscolo)
- "carol" (minuscolo)

La list comprehension assegnata a `uppercase_names` (nota: corretto il nome della variabile da `upper_case` a `uppercase_names` per coerenza con l'esempio originale) si compone di:

- Centro(nucleo): `for name in names` ; cioè "per ogni elemento (che chiameremo `name`) nella lista `names` ..."
- Output: `name.upper()` ; cioè "converti/restituisci l'elemento in maiuscolo"
Traduzione: "Per ogni elemento nella lista `names` , restituisci il nome in lettere maiuscole"

Esempio 3: Appiattare una lista di liste

Ipotizziamo che si abbia una matrice di numeri (in Python lista di liste) e che si voglia appiattare la matrice ritornando una lista.



Python

```
1 matrix:list[int] = [[1,2], [3,4]]
2 flat = [num for row in matrix for num in row]
3 print(flat)
```

Abbiamo la matrice di partenza:

```
matrix = [[1,2],[3,4]]
```

Dopodiché la list comprehension viene assegnata alla variabile `flat` e si compone di:

- Primo Nucleo(centro):
`for row in matrix` ; cioè "Per ogni riga nella lista `matrix` ..."
- Secondo nucleo:
`for num in row` ; cioè "...per ogni numero itera sulla riga..."
- Output:
`num` ;cioè "...aggiungi ogni numero alla lista risultante, cioè `flat` "

Traduzione:

"Per ogni riga nella matrice, per ogni numero in quella riga, metti il numero nella lista `flat` ".

In questo caso, essendo una list comprehension annidata con due cicli for, l'ordine dei cicli è cruciale: prima si scorrono le righe (`row`), poi i valori dentro ogni riga (`num`).

Inoltre non serve alcuna condizione `if` , poiché si vuole conservare tutti gli elementi, quindi in questo caso il filtro non viene neanche definito nella list comprehension

Set e dictionary Comprehension

Come abbiamo detto le comprehension si usano anche con i [set](#) e i [dizionari](#).
La sintassi è uguale cambiano solo le parentesi.

Esempio 1: Set Comprehension

mettiamo che abbiamo un set di stringhe e vogliamo che vogliamo la lunghezza di ogni elemento:



Python

```
1 words = {"hi", "hello", "hey", "hi"} # Set di stringhe (elementi unici)
2 unique_lengths = {len(word) for word in words} # Set comprehension
3 print(unique_lengths) # Output: {2, 3, 5}
```

Abbiamo un set originale:

- `words= {"hi","hello", "hey", "hi"}`

Essendo un set, quindi gli elementi sono unici e non si ammettono duplicati, il secondo "hi" viene ignorato.

La set comprehension viene assegnata alla variabile `unique_lengths`, e si compone di:

- Nucleo(centro): `for word in words`; cioè "Per ogni elemento (chiamato `word`) nel set `words` ..."
- Output: `len(word)`: cioè "...calcola la lunghezza della stringa `word`"
- Le parentesi graffe `{}` creano un nuovo set (senza duplicati);
"...e raccogli i risultati in un nuovo set (senza duplicati)"

Traduzione: "Per ogni parola nel set originale, calcola la sua lunghezza e raccogli i risultati in un nuovo set"

Mentre per quanto riguarda i dizionari:



Python

```
1 squares_dict = {x:x**2 for x in range(5)}
2 print(squares_dict)
```

Esattamente come per la list comprehension e il set comprehension ma questo genera dizionari.

La sintassi è :

```
1 {chiave: valore for elemento in iterabile}
```

Ovvero

- **Nucleo (centro):**

```
for x in range(5)
```

→ "Per ogni elemento *x* nel range da 0 a 4..."

- **Output (coppia chiave-valore):**

```
x: x**2 :
```

- "Crea una coppia dove:
- Chiave: *x* (numero originale)*
- Valore: *x**2* (quadrato del numero)"

Traduzione: "Per ogni numero *x* da 0 a 4, crea una coppia chiave-valore dove la chiave è *x*, e il valore è il suo quadrato"

Quando Non Usare le Comprehension in Python

1. Evita l'uso eccessivo: la leggibilità viene prima!

Le comprehension sono potenti, ma **non sempre sono la scelta più leggibile**.

- **Usale quando** rendono il codice più chiaro e conciso.
- **Evitale quando** il codice diventa difficile da capire a prima vista.

Esempio (da evitare):



Python

```
1 # Poco leggibile con troppe operazioni
2 result = [x**2 if x % 2 == 0 else x**3 for x in range(10) if x >
3           2 and x < 8]
```

Meglio usare un `for` tradizionale:



Python

```
1 result = []
2 for x in range(10):
3     if 2 < x < 8:
4         result.append(x**2 if x % 2 == 0 else x**3)
```

2. Non usarle per effetti collaterali (side-effects)

Le comprehension sono pensate per **creare nuove liste/set/dizionari**, non per:

- Stampare (`print`)
- Modificare variabili esterne
- Eseguire operazioni I/O (es. aprire file)

Esempio sbagliato (stampa in una comprehension):



Python

```
1  # Non usare una comprehension solo per stampare!
2  [print(x) for x in range(5)]  # Funziona, ma è anti-pattern!
```

Alternativa corretta (ciclo `for` esplicito):



Python

```
1  for x in range(5):
2      print(x)  # Molto più chiaro!
```

3. Le comprehension annidate possono diventare illeggibili

Più di **due livelli di annidamento** rendono il codice difficile da mantenere.

Esempio complicato (comprehension annidata)



Python

```
1  matrix = [[1, 2], [3, 4]]
2  flat_squares = [x**2 for row in matrix for x in row if x % 2 ==
3                  0]
```

Alternativa più chiara (cicli espliciti)



Python

```
1  flat_squares = []
2  for row in matrix:
3      for x in row:
```

```
4     if x % 2 == 0:
5         flat_squares.append(x**2)
```

Quando invece sono perfette?

- ✓ Trasformazioni semplici (es. filtrare o mappare valori)
- ✓ Operazioni su singola riga (senza troppa logica condizionale)
- ✓ Creare strutture dati in modo pulito (liste, set, dizionari)

Regola d'oro

"Scrivi codice come se fosse letto da qualcuno che non lo ha mai visto prima."

Se una comprehension diventa troppo complessa, **usa un ciclo** `for` tradizionale.